Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 23 - PAO 2024 – II

Agenda

- Búsqueda heurística
  - Best - First
  - Teorema de Admisibilidad
  - Algoritmo de backtracking
- Otras formas de representación de conocimiento

Búsqueda heurística: Admisibilidad

- Recordar la función: f (n) = g (n) + h (n)

Dónde:

- *n* es el estado actual (o concurrente) durante la búsqueda
- *g* es el costo desde el estado inicial hasta el estado actual *n*
- *h* es el valor heurístico o el costo adicional para alcanzar la meta desde el estado actual *n*

- Si esta función se usa con el algoritmo de búsqueda primero el mejor, entonces se llama algoritmo tipo A.
- Cuando un algoritmo heurístico garantiza encontrar una solución óptima, si existe, entonces el algoritmo se llama Admisible, A.

Búsqueda heurística: Algoritmo A*

Considerar:

$$f^*(n) = g^*(n) + h^*(n)$$

Dónde:

- $g^*(n)$ = costo de la ruta más corta desde el estado inicial hasta n.
- $h^*(n)$ = costo de la ruta más corta desde n hasta la meta.

- Si un algoritmo A, con una función de evaluación f^*, donde $h^*(n) < h(n)$, entonces el algoritmo A se llama A*

Búsqueda heurística

- **A* Teorema de Admisibilidad**

  - Un algoritmo A*, que usa h*(n), donde h*(n) < h(n), para todo n, entonces el algoritmo garantiza encontrar una ruta óptima desde n hasta el objetivo, si esta ruta existe.
  - Primero a lo ancho se puede considerar como un algoritmo A*, en el que f(n) = g(n) + 0

- Recuerdan el algoritmo Best-First?

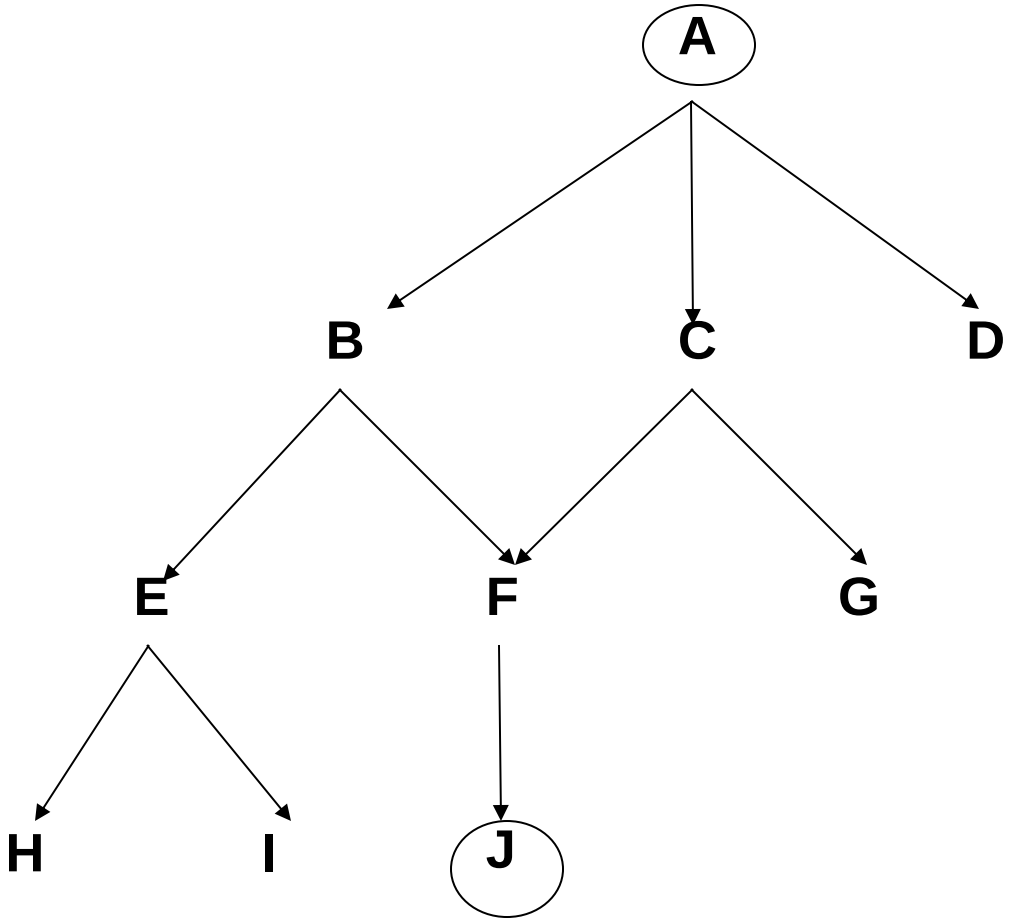
```
function best_first_search;

begin
  open := [Start];                                          % initialize
  closed := [ ];
  while open ≠ [ ] do                                       % states remain
    begin
      remove the leftmost state from open, call it X;
      if X = goal then return the path from Start to X
      else begin
            generate children of X;
            for each child of X do
            case
                the child is not on open or closed:
                  begin
                      assign the child a heuristic value;
                      add the child to open
                  end;
                the child is already on open:
                  if the child was reached by a shorter path
                  then give the state on open the shorter path
                the child is already on closed:
                  if the child was reached by a shorter path then
                    begin
                      remove the state from closed;
                      add the child to open
                    end;
            end;                                            % case
            put X on closed;
            re-order states on open by heuristic merit (best leftmost)
          end;
return FAIL                                                 % open is empty
end.
```

Backtrack algorithm

```
begin
    SL:=[start]; NSL:=[start]; DE:=[ ]; CS:= start;
    While NSL π [ ] do
        begin
            if CS:= goal (or meets the goal description)
                then return SL;
            if CS has no children (excluding the nodes already in DE, SL and NSL)
                then
                    begin
                        while SL not empty and CS = first element of SL do
                            begin
                                add CS to DE;
                                remove first element of SL;
                                remove first element of NSL;
                                CS := first element of NSL;
                            end
                        add CS a SL;
                    end
                else
                    begin
                        place the children of CS (except if already in DE, SL or NSL) in NSL;
                        CS := first element of NSL;
                        add CS a SL
                    end
        end;
    return FAIL;
end.
```

Backtrack algorithm

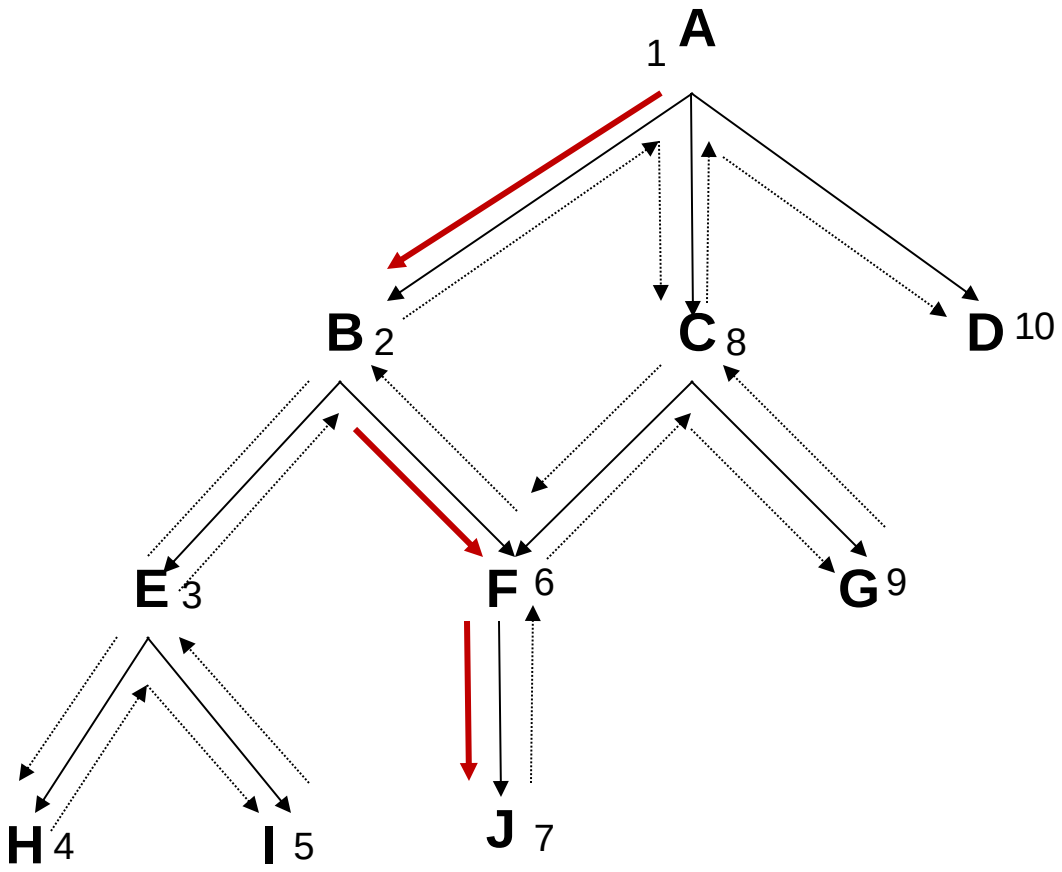


Backtrack algorithm

SL = [A]; NSL = [A]; DE = [ ]; CS = A;

While 1	**While 2**	**Variable**	**List**	**List**	**List**
NSL ≠ ∅	**SL ≠ ∅** **CS = SL**	**CS**	**SL**	**NSL**	**DE**
0	-	A	[A]	[A]	[ ]
1	-	B	[BA]	[BCDA]	[ ]
2	-	E	[EBA]	[EFBCDA]	[ ]
3	-	H	[HEBA]	[HIEFBCDA]	[ ]
4	0	I	[EBA]	[IEFBCDA]	[H]
	1	I	[IEBA]	[IEFBCDA]	[H]
5	0	E	[EBA]	[EFBCDA]	[IH]
	1	F	[BA]	[FBCDA]	[EIH]
	2	F	[FBA]	[FBCDA]	[EIH]
6	-	J	[JFBA]	[JFBCDA]	[EIH]
7	-	J	**[JFBA]**	[JFBCDA]	[EIH]

Backtrack algorithm



Representación del conocimiento

- El método debe ser:
  - Expresivo
    - Permitir representar toda la información necesaria y la experiencia.
    - Proporcionar un esquema natural para representar el conocimiento requerido.
  - Eficiente
    - Para permitir la ejecución eficiente de código en una computadora.
    - En general permitir resolver no solo problemas cuantitativos, sino también cualitativos.
    - Organizar grandes y diferentes tipos de conocimiento, y permitir implementar algoritmos de búsqueda.

Representación del conocimiento

- Esquemas de representación del conocimiento:

1. Representación lógica:
   - Utiliza expresiones lógicas formales para representar el conocimiento.
   - Cálculo del predicado de primer orden.

2. Representación procedimental:
   - El conocimiento se representa como un conjunto de instrucciones.
   - Reglas tipo: SI - ENTONCES.

Representación del conocimiento

3. Representación en red:

   El conocimiento se captura en forma de grafos.

   Los nodos representan objetos o conceptos, y los enlaces representan relaciones o asociaciones entre ellos.

   RN, redes semánticas, gráficos conceptuales, mapas cognitivos.

4. Representación estructurada:

   Esta es una extensión de la representación de la red.

   Los nodos son estructuras complejas que contienen "ranuras con valores asociados"

   Scripts, frames, objetos.